```
# Set Keras backend to 'torch' if TensorFlow is not available, or default
import os
if "KERAS_BACKEND" not in os.environ:
    os.environ["KERAS_BACKEND"] = "torch"

# Core Data Science Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-Learn for Preprocessing & Multi-Class Evaluation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

# Deep Learning with Keras
import keras
from keras import layers, models, callbacks

# Set random seeds for classroom reproducibility
np.random.seed(42)
keras.utils.set_random_seed(42)

# Visualization styling
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

print("=" * 60)
print(f"✅ Keras Version : {keras.__version__}")
print(f"✅ Backend Engine: {keras.backend.backend()}")
print(f"✅ Pandas Version: {pd.__version__}")
print(f"✅ NumPy Version : {np.__version__}")
print("=" * 60)
```

```
============================================================
✅ Keras Version : 3.13.2
✅ Backend Engine: torch
✅ Pandas Version: 2.2.3
✅ NumPy Version : 2.1.3
============================================================
```

```
data_path = "forest_cover_type.csv"

# Load local 100,000-row dataset
df = pd.read_csv(data_path)

print(f"Successfully loaded dataset: {df.shape[0]:,} rows × {df.shape[1]} columns")
df.head()
```

```
# Mapping of integer class codes to ecological tree species names
COVER_TYPE_NAMES = {
    1: 'Spruce/Fir',
    2: 'Lodgepole Pine',
    3: 'Ponderosa Pine',
    4: 'Cottonwood/Willow',
```

```
    5: 'Aspen',
    6: 'Douglas-fir',
    7: 'Krummholz'
}

# Create a human-readable label column for EDA
df['Cover_Type_Name'] = df['Cover_Type'].map(COVER_TYPE_NAMES)

print("Class Distribution in 100,000 Records:")
class_summary = pd.DataFrame({
    'Class ID': list(COVER_TYPE_NAMES.keys()),
    'Species Name': list(COVER_TYPE_NAMES.values()),
    'Count': [df['Cover_Type'].value_counts().get(i, 0) for i in range(1, 8)],
    'Percentage (%)': [(df['Cover_Type'].value_counts(normalize=True).get(i, 0) * 100) for i in range(1, 8)]
})
display(class_summary)
```

```
Class Distribution in 100,000 Records:
```

	Class ID	Species Name	Count	Percentage (%)
0	1	Spruce/Fir	36460	36.460
1	2	Lodgepole Pine	48760	48.760
2	3	Ponderosa Pine	6154	6.154
3	4	Cottonwood/Willow	473	0.473
4	5	Aspen	1634	1.634
5	6	Douglas-fir	2989	2.989
6	7	Krummholz	3530	3.530

Next steps: Generate code with `class_summary` New interactive sheet

3. Data Inspection & CleaningLet's inspect the data types, check for nulls, and verify feature ranges.

```
# 1. Check for missing values
print(f"Total null values across all columns: {df.isnull().sum().sum()}")

# 2. Inspect continuous feature ranges
continuous_features = [
    'Elevation', 'Aspect', 'Slope', 'Horizontal_Distance_To_Hydrology',
    'Vertical_Distance_To_Hydrology', 'Horizontal_Distance_To_Roadways',
    'Hillshade_9am', 'Hillshade_Noon', 'Hillshade_3pm',
    'Horizontal_Distance_To_Fire_Points'
]

print("\nSummary statistics of key continuous features:")
df[continuous_features].describe().T[['min', 'mean', '50%', 'max', 'std']]
```

```
Total null values across all columns: 0

Summary statistics of key continuous features:
```

	min	mean	50%	max	std
Elevation	1872.0	2960.09612	2997.0	3856.0	279.544664
Aspect	0.0	155.71107	127.0	360.0	111.803239
Slope	0.0	14.12481	13.0	61.0	7.505089
Horizontal_Distance_To_Hydrology	0.0	269.43533	218.0	1397.0	212.229191
Vertical_Distance_To_Hydrology	-163.0	46.64551	30.0	598.0	58.501250
Horizontal_Distance_To_Roadways	0.0	2342.71861	1994.0	7063.0	1554.001595
Hillshade_9am	0.0	212.10793	218.0	254.0	26.859045
Hillshade_Noon	0.0	223.32662	226.0	254.0	19.753808
Hillshade_3pm	0.0	142.55599	143.0	253.0	38.380120
Horizontal_Distance_To_Fire_Points	0.0	1977.14222	1710.0	7172.0	1317.499106

```
fig, ax = plt.subplots(figsize=(12, 5))

species_order = [COVER_TYPE_NAMES[i] for i in range(1, 8)]
palette = sns.color_palette("tab10", 7)

sns.countplot(
    data=df, x='Cover_Type_Name', order=species_order, palette=palette, ax=ax
)
ax.set_title("Distribution of 7 Forest Cover Classes (N = 100,000)", fontsize=14, fontweight='bold')
ax.set_xlabel("Forest Tree Species", fontsize=12)
ax.set_ylabel("Record Count", fontsize=12)
plt.xticks(rotation=20, ha='right')

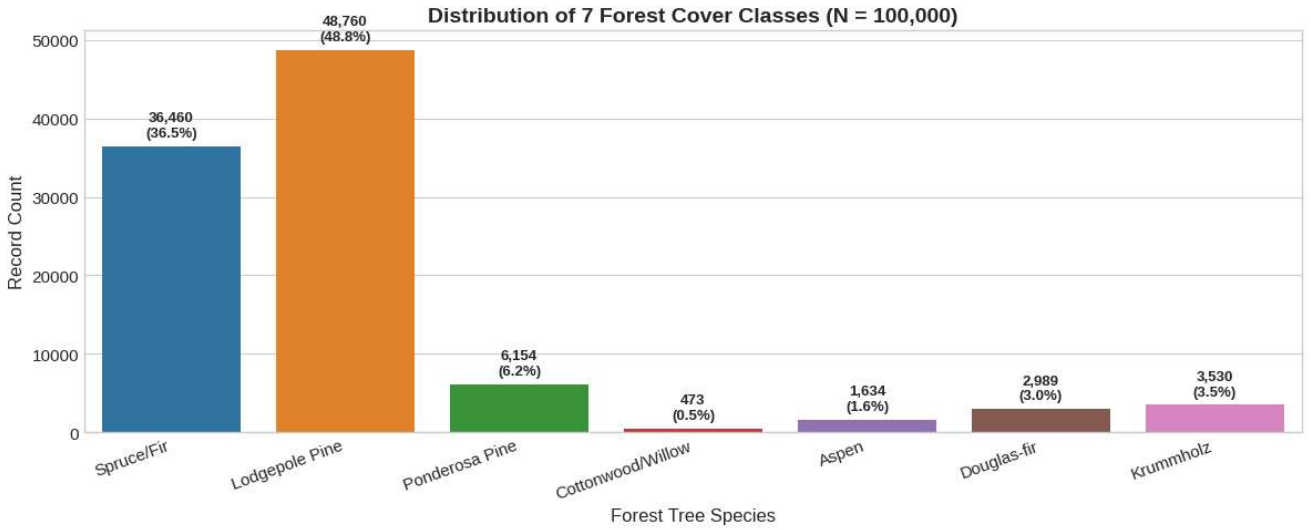
# Add percentage annotations
total_rows = len(df)
for p in ax.patches:
    height = p.get_height()
    pct = (height / total_rows) * 100
    ax.annotate(f"{height:,.0f}\n({pct:.1f}%)",
                (p.get_x() + p.get_width() / 2., height + 800),
                ha='center', va='bottom', fontsize=9.5, fontweight='bold')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_1634/462466953.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.countplot(
```



4.2 Ecological Feature Analysis: Elevation vs. Tree SpeciesIn mountainous ecosystems, **Elevation** is the single strongest physical factor determining which tree species can survive.

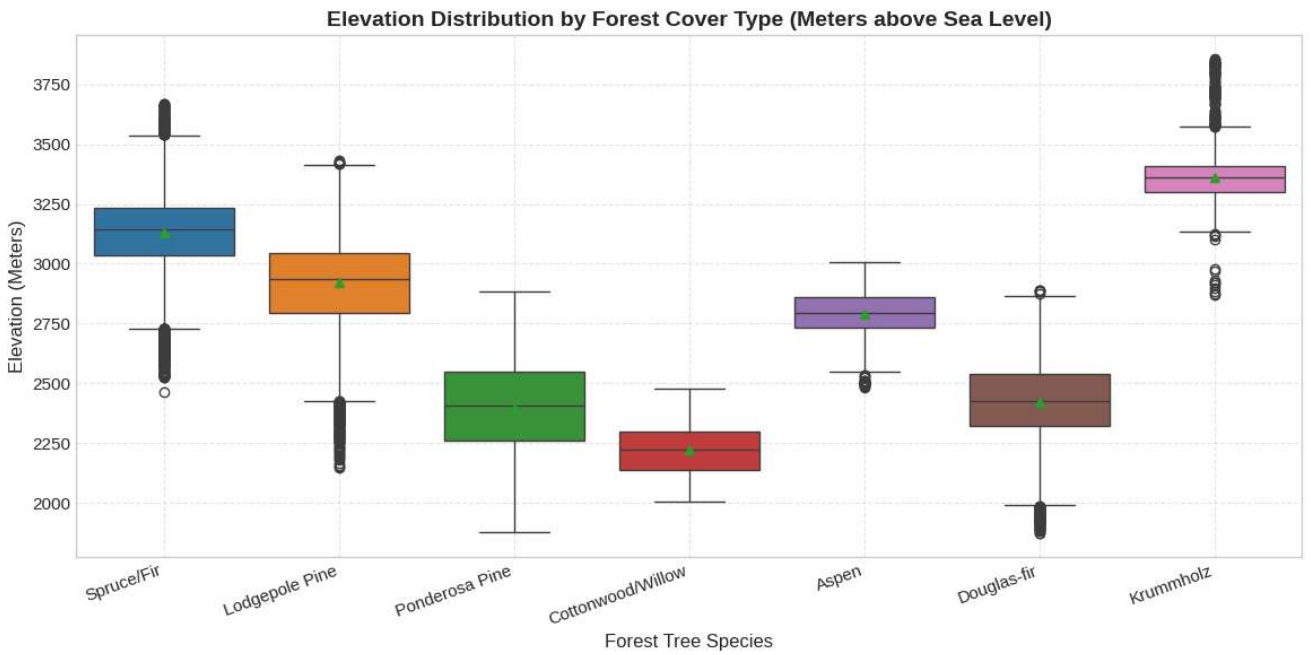
```
plt.figure(figsize=(14, 6))

sns.boxplot(
    data=df, x='Cover_Type_Name', y='Elevation',
    order=species_order, palette=palette, showmeans=True
)
plt.title("Elevation Distribution by Forest Cover Type (Meters above Sea Level)", fontsize=14, fontweight='bold')
plt.xlabel("Forest Tree Species", fontsize=12)
plt.ylabel("Elevation (Meters)", fontsize=12)
plt.xticks(rotation=20, ha='right')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

```
/tmp/ipykernel_1634/391802677.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(
```



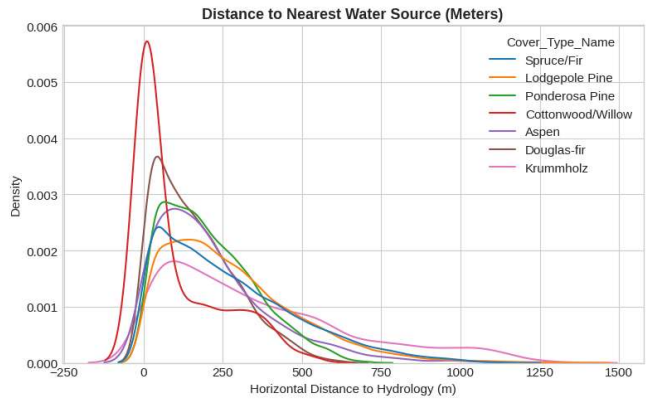
4.3 Distance to Hydrology and Roadways

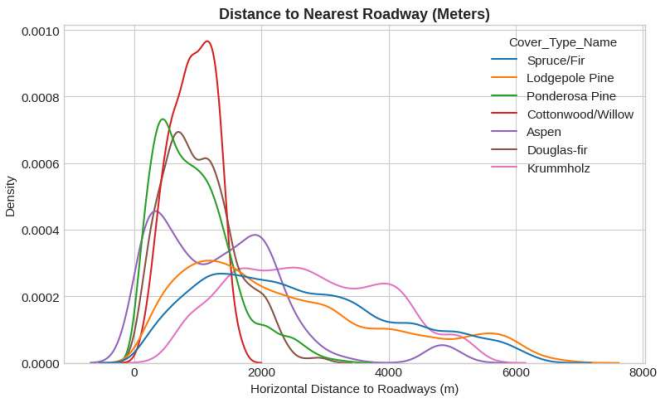
```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Distance to Water
sns.kdeplot(
    data=df, x='Horizontal_Distance_To_Hydrology', hue='Cover_Type_Name',
    hue_order=species_order, common_norm=False, palette=palette, ax=axes[0]
)
axes[0].set_title("Distance to Nearest Water Source (Meters)", fontsize=13, fontweight='bold')
axes[0].set_xlabel("Horizontal Distance to Hydrology (m)")

# Distance to Roadways
sns.kdeplot(
    data=df, x='Horizontal_Distance_To_Roadways', hue='Cover_Type_Name',
    hue_order=species_order, common_norm=False, palette=palette, ax=axes[1]
)
axes[1].set_title("Distance to Nearest Roadway (Meters)", fontsize=13, fontweight='bold')
axes[1].set_xlabel("Horizontal Distance to Roadways (m)")

plt.tight_layout()
plt.show()
```






```
# Separate features and target
X = df.drop(columns=['Cover_Type', 'Cover_Type_Name'])

# Convert 1-indexed labels (1..7) to 0-indexed labels (0..6)
y = (df['Cover_Type'] - 1).values

# 80/20 Stratified Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(f"Training set : {X_train.shape[0]:,} samples, {X_train.shape[1]} features")
print(f"Testing set  : {X_test.shape[0]:,} samples, {X_test.shape[1]} features")
print(f"Unique classes in y_train: {np.unique(y_train)} (Count: {len(np.unique(y_train))})")
```

```
Training set : 80,000 samples, 54 features
Testing set  : 20,000 samples, 54 features
Unique classes in y_train: [0 1 2 3 4 5 6] (Count: 7)
```

```
scaler = StandardScaler()

# Fit strictly on train, then transform both
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Verify scaled statistics
scale_check = pd.DataFrame({
    'Raw Feature Mean': X_train.iloc[:, :5].mean(),
    'Raw Feature Std': X_train.iloc[:, :5].std(),
    'Scaled Feature Mean': X_train_scaled[:, :5].mean(axis=0).round(4),
    'Scaled Feature Std': X_train_scaled[:, :5].std(axis=0).round(4)
})
display(scale_check)
```

	Raw Feature Mean	Raw Feature Std	Scaled Feature Mean	Scaled Feature Std
Elevation	2960.048325	279.379255	-0.0	1.0
Aspect	155.689425	111.874383	0.0	1.0
Slope	14.117662	7.500526	0.0	1.0
Horizontal_Distance_To_Hydrology	269.841100	212.146653	0.0	1.0
Vertical_Distance_To_Hydrology	46.678787	58.468540	0.0	1.0

Next steps: Generate code with `scale_check` New interactive sheet

```
num_features = X_train_scaled.shape[1]
num_classes = 7

model = models.Sequential([
    # Input Layer
    layers.Input(shape=(num_features,)),

    # Hidden Layer 1
    layers.Dense(128, activation='relu', name='dense_1'),
    layers.BatchNormalization(name='batch_norm_1'),
    layers.Dropout(0.25, name='dropout_1'),

    # Hidden Layer 2
    layers.Dense(64, activation='relu', name='dense_2'),
    layers.Dropout(0.20, name='dropout_2'),

    # Hidden Layer 3
    layers.Dense(32, activation='relu', name='dense_3'),

    # Multi-Class Output Layer (7 units with Softmax)
    layers.Dense(num_classes, activation='softmax', name='output_softmax')
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 128)	7,040
batch_norm_1 (BatchNormalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dropout_2 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 32)	2,080
output_softmax (Dense)	(None, 7)	231

Total params: 18,119 (70.78 KB)

```
# Compile model
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Early stopping callback
early_stopping = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=6,
    restore_best_weights=True,
    verbose=1
)

EPOCHS = 50
BATCH_SIZE = 128

print("Starting Multi-Class Training on 80,000 environmental records...")
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.20,  # 20% of training set for validation monitoring
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stopping],
    verbose=1
)
print("✅ Multi-Class Training Complete!")
```

```
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8173 - loss: 0.4303 - val_accuracy: 0.8496 - val_loss: 0.3673
Epoch 25/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8181 - loss: 0.4287 - val_accuracy: 0.8554 - val_loss: 0.3632
Epoch 26/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.8175 - loss: 0.4291 - val_accuracy: 0.8516 - val_loss: 0.3703
Epoch 27/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.8196 - loss: 0.4285 - val_accuracy: 0.8543 - val_loss: 0.3645
Epoch 28/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8195 - loss: 0.4271 - val_accuracy: 0.8544 - val_loss: 0.3628
Epoch 29/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8179 - loss: 0.4305 - val_accuracy: 0.8521 - val_loss: 0.3685
Epoch 30/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.8171 - loss: 0.4276 - val_accuracy: 0.8546 - val_loss: 0.3647
Epoch 31/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8184 - loss: 0.4288 - val_accuracy: 0.8496 - val_loss: 0.3737
Epoch 32/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8183 - loss: 0.4274 - val_accuracy: 0.8538 - val_loss: 0.3641
Epoch 33/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8191 - loss: 0.4252 - val_accuracy: 0.8548 - val_loss: 0.3631
Epoch 34/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.8189 - loss: 0.4287 - val_accuracy: 0.8554 - val_loss: 0.3620
Epoch 35/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8201 - loss: 0.4254 - val_accuracy: 0.8546 - val_loss: 0.3607
Epoch 36/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8194 - loss: 0.4251 - val_accuracy: 0.8551 - val_loss: 0.3609
Epoch 37/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.8197 - loss: 0.4247 - val_accuracy: 0.8546 - val_loss: 0.3606
Epoch 38/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.8205 - loss: 0.4234 - val_accuracy: 0.8554 - val_loss: 0.3643
Epoch 39/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8205 - loss: 0.4229 - val_accuracy: 0.8565 - val_loss: 0.3590
Epoch 40/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8204 - loss: 0.4263 - val_accuracy: 0.8548 - val_loss: 0.3600
Epoch 41/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.8188 - loss: 0.4279 - val_accuracy: 0.8548 - val_loss: 0.3647
Epoch 42/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8195 - loss: 0.4250 - val_accuracy: 0.8551 - val_loss: 0.3557
Epoch 43/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8212 - loss: 0.4210 - val_accuracy: 0.8525 - val_loss: 0.3611
Epoch 44/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8205 - loss: 0.4223 - val_accuracy: 0.8556 - val_loss: 0.3563
Epoch 45/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.8214 - loss: 0.4241 - val_accuracy: 0.8585 - val_loss: 0.3547
Epoch 46/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8211 - loss: 0.4221 - val_accuracy: 0.8569 - val_loss: 0.3552
Epoch 47/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8229 - loss: 0.4224 - val_accuracy: 0.8589 - val_loss: 0.3567
Epoch 48/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8227 - loss: 0.4203 - val_accuracy: 0.8599 - val_loss: 0.3529
Epoch 49/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.8239 - loss: 0.4173 - val_accuracy: 0.8584 - val_loss: 0.3543
Epoch 50/50
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.8192 - loss: 0.4213 - val_accuracy: 0.8567 - val_loss: 0.3548
Restoring model weights from the end of the best epoch: 48.
✅ Multi-Class Training Complete!
```

9. Visualizing Multi-Class Training & Validation Curves

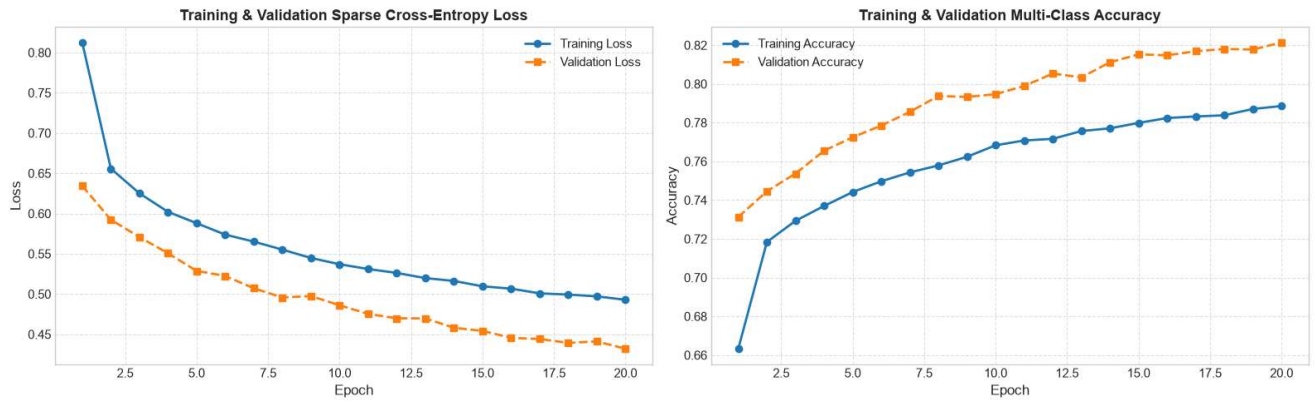
```
history_dict = history.history
epochs_range = range(1, len(history_dict['loss']) + 1)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Loss Curve
axes[0].plot(epochs_range, history_dict['loss'], 'o-', label='Training Loss', color='#1f77b4', linewidth=2)
axes[0].plot(epochs_range, history_dict['val_loss'], 's--', label='Validation Loss', color='#ff7f0e', linewidth=2)
axes[0].set_title('Training & Validation Sparse Cross-Entropy Loss', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss', fontsize=12)
axes[0].legend(fontsize=11)
axes[0].grid(True, linestyle='--', alpha=0.6)

# Accuracy Curve
axes[1].plot(epochs_range, history_dict['accuracy'], 'o-', label='Training Accuracy', color='#1f77b4', linewidth=2)
axes[1].plot(epochs_range, history_dict['val_accuracy'], 's--', label='Validation Accuracy', color='#ff7f0e', linewidth=2)
axes[1].set_title('Training & Validation Multi-Class Accuracy', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Accuracy', fontsize=12)
axes[1].legend(fontsize=11)
axes[1].grid(True, linestyle='--', alpha=0.6)
```

```
plt.tight_layout()
plt.show()
```




```
# Predict probability distributions
y_pred_probs = model.predict(X_test_scaled, verbose=0)

# Extract highest-probability class index (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# Compute metrics
overall_acc = accuracy_score(y_test, y_pred)
macro_f1 = f1_score(y_test, y_pred, average='macro')
weighted_f1 = f1_score(y_test, y_pred, average='weighted')

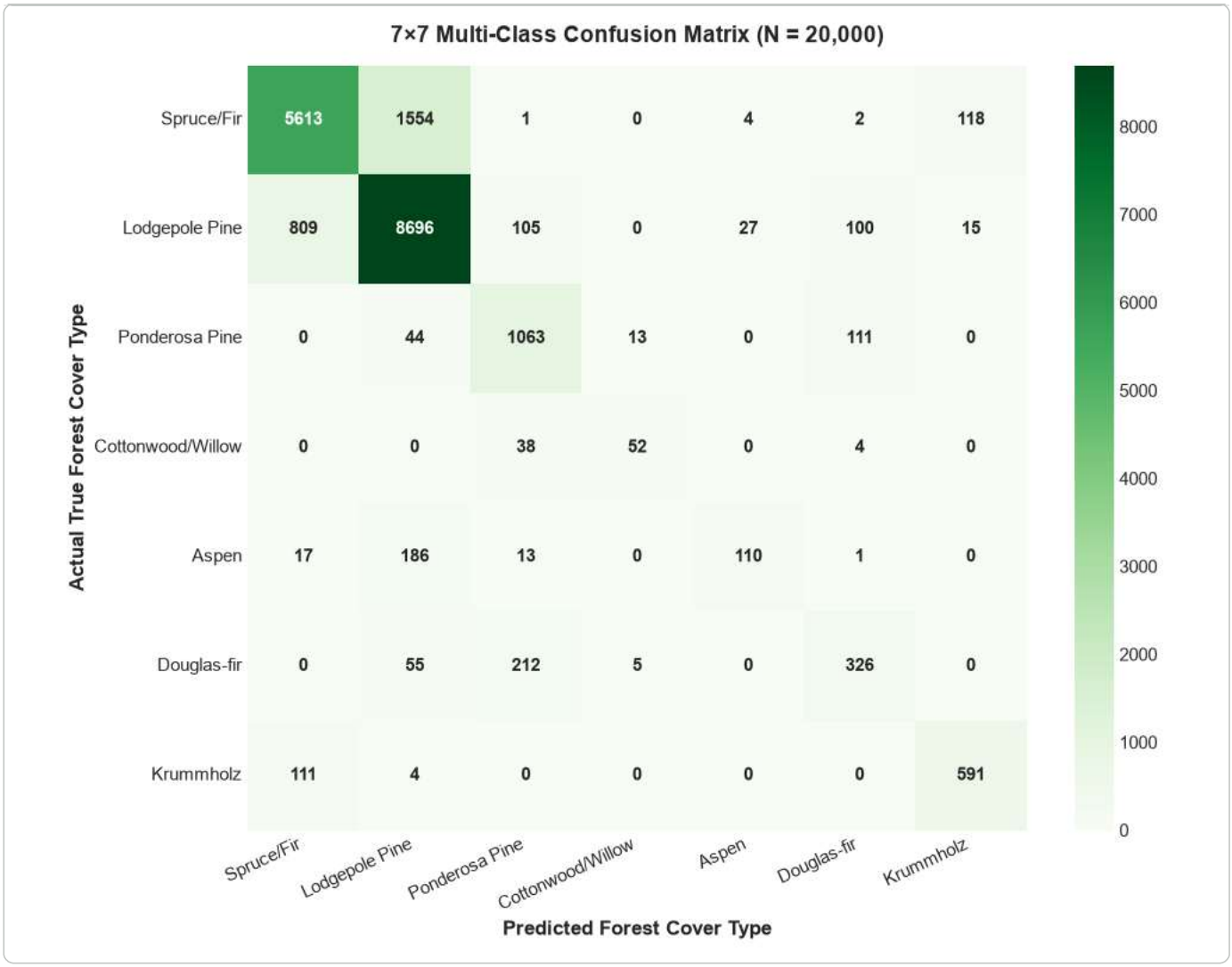
print("=" * 65)
print(f"MULTI-CLASS TEST SET EVALUATION (N = {len(y_test):,} Records)")
print("=" * 65)
print(f"Overall Accuracy : {overall_acc * 100:.2f}%")
print(f"Macro F1-Score   : {macro_f1 * 100:.2f}%")
print(f"Weighted F1-Score: {weighted_f1 * 100:.2f}%")
print("=" * 65)
```

```
=================================================================
📊 MULTI-CLASS TEST SET EVALUATION (N = 20,000 Records)
=================================================================
Overall Accuracy : 82.25%
Macro F1-Score   : 70.97%
Weighted F1-Score: 81.97%
=================================================================
```

10.1 7×7 Multi-Class Confusion Matrix Heatmap

```
cm = confusion_matrix(y_test, y_pred)
class_labels = [COVER_TYPE_NAMES[i] for i in range(1, 8)]

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Greens',
    xticklabels=class_labels, yticklabels=class_labels,
    cbar=True, annot_kws={'size': 11, 'weight': 'bold'}
)
plt.title(f'7×7 Multi-Class Confusion Matrix (N = {len(y_test):,})', fontsize=14, fontweight='bold', pad=14)
plt.xlabel('Predicted Forest Cover Type', fontsize=12, fontweight='bold')
plt.ylabel('Actual True Forest Cover Type', fontsize=12, fontweight='bold')
plt.xticks(rotation=25, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



10.2 Detailed Classification Report (Per-Species Metrics)

```
# Classification Report for all 7 classes
print("Detailed Per-Class Performance Report:")
print(classification_report(y_test, y_pred, target_names=class_labels, digits=4))
```

```
Detailed Per-Class Performance Report:
                  precision    recall  f1-score   support

       Spruce/Fir     0.8569    0.7697    0.8110      7292
   Lodgepole Pine     0.8251    0.8917    0.8571      9752
   Ponderosa Pine     0.7423    0.8635    0.7983      1231
Cottonwood/Willow     0.7429    0.5532    0.6341        94
            Aspen     0.7801    0.3364    0.4701       327
      Douglas-fir     0.5993    0.5452    0.5709       598
         Krummholz     0.8163    0.8371    0.8266       706

         accuracy                         0.8226     20000
        macro avg     0.7661    0.6853    0.7097     20000
     weighted avg     0.8234    0.8226    0.8197     20000
```

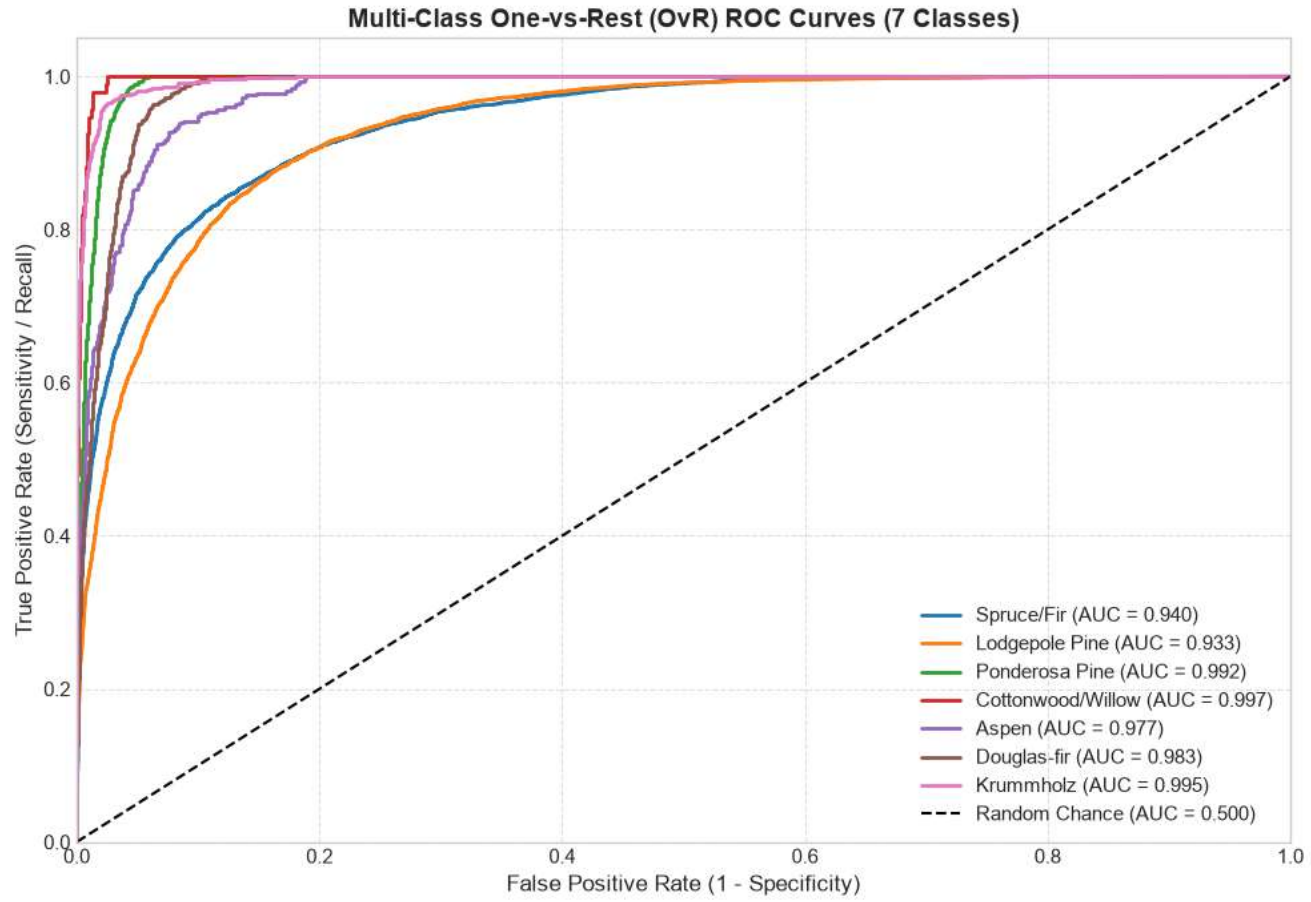
```
# Binarize labels for multi-class ROC curve
y_test_bin = label_binarize(y_test, classes=range(7))

plt.figure(figsize=(10, 7))
colors = sns.color_palette("tab10", 7)

# Plot ROC curve for each class
for i in range(7):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_pred_probs[:, i])
    auc_score = roc_auc_score(y_test_bin[:, i], y_pred_probs[:, i])
    plt.plot(
```

```
            fpr, tpr, color=colors[i], lw=2,
            label=f'{class_labels[i]} (AUC = {auc_score:.3f})'
        )

    plt.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Chance (AUC = 0.500)')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
    plt.ylabel('True Positive Rate (Sensitivity / Recall)', fontsize=12)
    plt.title('Multi-Class One-vs-Rest (OvR) ROC Curves (7 Classes)', fontsize=14, fontweight='bold')
    plt.legend(loc='lower right', fontsize=10.5)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.show()
```




```
def predict_forest_cover(sample_df, model, scaler, feature_names):
    """
    Multi-Class Inference Pipeline:
    Scales raw environmental features, predicts full 7-class probability distribution,
    and returns top-3 most probable tree species.
    """
    # Align columns
    aligned_df = sample_df.reindex(columns=feature_names, fill_value=0.0)

    # Scale features
    scaled_features = scaler.transform(aligned_df)

    # Predict probabilities (N, 7)
    pred_probs = model.predict(scaled_features, verbose=0)

    print("=" * 75)
    print("🌲 REAL-TIME MULTI-CLASS INFERENCE (Top-3 Predicted Species)")
    print("=" * 75)

    for idx, probs in enumerate(pred_probs):
        top3_indices = np.argsort(probs)[::-1][:3]
        top_class = COVER_TYPE_NAMES[top3_indices[0] + 1]

        print(f"\nTerrain Location #{idx + 1}:")
```

```
        print(f"  • Primary Predicted Species: 🌲 {top_class} ({probs[top3_indices[0]] * 100:.2f}% confidence)")
        print(f"  • Full Top-3 Probabilities:")
        for rank, c_idx in enumerate(top3_indices, 1):
            s_name = COVER_TYPE_NAMES[c_idx + 1]
            bar = '█' * int(probs[c_idx] * 25)
            print(f"     {rank}. {s_name:<20}: {probs[c_idx] * 100:5.2f}% | {bar}")
        print("-" * 75)

# Select 3 representative test samples from different elevation profiles
sample_locations = X_test.iloc[[0, 15, 30]].copy()
predict_forest_cover(sample_locations, model, scaler, X_train.columns.tolist())
```

```
================================================================================
🔮 REAL-TIME MULTI-CLASS INFERENCE (Top-3 Predicted Species)
================================================================================

Terrain Location #1:
  • Primary Predicted Species: 🌲 Lodgepole Pine (98.52% confidence)
  • Full Top-3 Probabilities:
     1. Lodgepole Pine      : 98.52% | ████████████████████████
     2. Spruce/Fir          :  1.18% |
     3. Aspen               :  0.24% |
---------------------------------------------------------------------------

Terrain Location #2:
  • Primary Predicted Species: 🌲 Spruce/Fir (84.91% confidence)
  • Full Top-3 Probabilities:
     1. Spruce/Fir          : 84.91% | █████████████████████
     2. Lodgepole Pine      : 13.50% | ███
     3. Krummholz           :  1.59% |
---------------------------------------------------------------------------

Terrain Location #3:
  • Primary Predicted Species: 🌲 Lodgepole Pine (81.07% confidence)
  • Full Top-3 Probabilities:
     1. Lodgepole Pine      : 81.07% | ████████████████████
```